Software Re-Engineering

Lecture 5

Preparing to refactor

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

Agenda

- Introduction of each layers of Software Development Life Cycle and Role of Reengineering
- Deciding whether to refactor or to rewrite from scratch
- Deciding what is and isn’t worth refactoring
- Methods for maintaining discipline when refactoring
- Common smells found in legacy code, and refactoring techniques for removing them
- Using automated tests to support refactoring

Introduction of each layers of Software Development Life Cycle and Role of Reengineering



Software Development Life Cycle (SDLC) is a systematic process that guides the development, maintenance, and management of software applications or systems.

1.Planning:

1. **Role:** In this phase, project objectives and requirements are defined. The feasibility of the project is assessed, and a project plan is created.
2. **Reengineering:** Reengineering can play a role in assessing the existing systems and determining whether they can be improved or need to be replaced entirely. This evaluation is essential in the planning phase to make informed decisions about whether to continue with the existing system or start from scratch.



1.Analysis:

1. **Role:** This phase involves a detailed analysis of the requirements. It defines what the software needs to do and how it will function.
2. **Reengineering:** In this phase, reengineering can be used to analyze the existing software, identify shortcomings, and gather insights into what needs to change to meet the new requirements. This can inform the creation of the new system or the modification of the existing one.

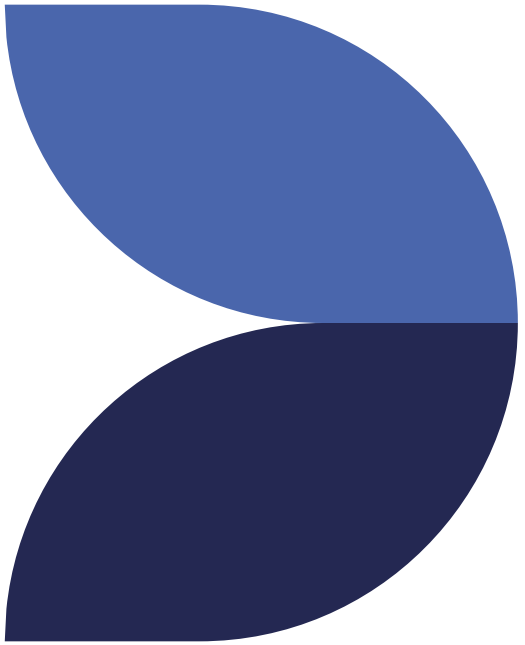
2.Design:

1. **Role:** The design phase involves creating a detailed blueprint of the software's architecture, including data structures, algorithms, and user interfaces.
2. **Reengineering:** Reengineering can guide the design of changes or enhancements to the existing system. It helps in designing the modifications required to align the existing system with the new design goals

1. **Implementation (Coding):**
    1. **Role:** This phase involves the actual coding of the software based on the design specifications.
    2. **Reengineering:** In some cases, reengineering may involve rewriting or refactoring portions of the existing code to improve maintainability or to integrate new functionality.
2. **Testing:**
    1. **Role:** Testing is the process of systematically verifying that the software functions correctly and meets the specified requirements.
    2. **Reengineering:** Reengineering may involve creating new test cases or modifying existing ones to ensure that the changes made to the software do not introduce new defects and that they meet the desired objectives.

1. **Deployment (Release):**
    1. **Role:** In this phase, the software is deployed to a production environment for end-users to access and use.
    2. **Reengineering:** Reengineering can involve transitioning from an old system to a new one. This transition must be carefully planned and executed to minimize disruptions to users.
2. **Maintenance and Support:**
    1. **Role:** This phase involves ongoing maintenance, bug fixes, updates, and support for the software throughout its lifecycle.
    2. **Reengineering:** Reengineering can play a critical role in ongoing maintenance by continually evaluating whether the software should be updated, enhanced, or replaced to keep it aligned with evolving business needs and technological advancements.

Decision time:
refactor or rewrite?



Modernization methods

- **Refactor:** Restructure and optimize the app's code to meet modern standards without changing its external behavior.
- **Rearchitect:** Create a new application architecture that enables improved performance and new capabilities.
- **Rewrite:** Rewrite the application from scratch, retaining its original scope and specifications.
- **Replace:** Completely eliminate the original application, and replace it with a new one. This option requires such an extreme investment, in terms of time, cost, and risk, that it is normally used only as a last resort.

The case against a rewrite

Risks

- unacceptably high bug count.
- taking longer to complete than originally planned, and thus overrun its budget.
- You might discover halfway through the project that the architecture is fundamentally unworkable and end up scrapping all the code you've written so far.
- Even worse, you might not discover these architectural weaknesses until you release the software to users, and it turns out to be totally unstable under load.

OVERHEAD

- build files, logging utilities, database access code, utilities to help with reading configuration files, and many other bits and pieces
- There’s also a checklist of administrative tasks to be performed, such as registering a new project on the issue tracker, setting up jobs on the continuous integration server, and maybe creating a new mailing list

ALWAYS TAKES LONGER THAN EXPECTED

- Estimating the size of a software project is notoriously difficult, and the larger the project, the more difficult it becomes.

Benefits of rewriting from scratch

Rewriting software from scratch also provides some benefits over refactoring.

FREEDOM

TESTABILITY

- A lot of legacy code has very few automated tests and was not designed with testability in mind.
- Once code has been written, it's very difficult to retroactively add testability, so writing tests for legacy code can take a prohibitive amount of developer time and effort.

When writing from scratch, on the other hand, you can build testability into the design from the start.

Necessary conditions for a rewrite

- **REFACTORING TRIED AND FAILED**

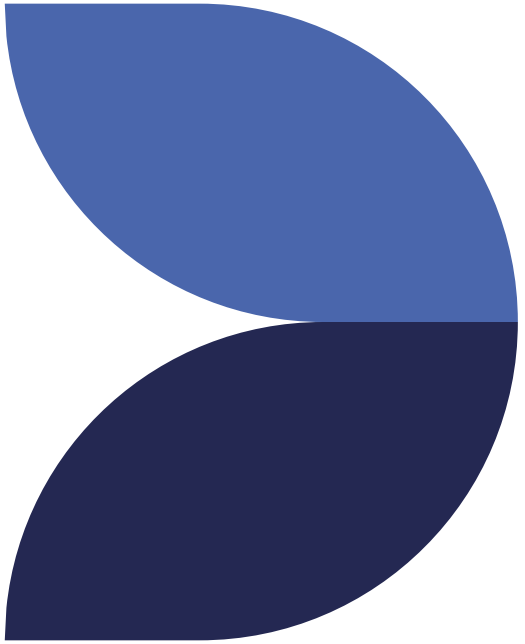
once you've spent a considerable amount of time and effort on refactoring, without a noticeable improvement in quality, is it time to start considering a full rewrite.

- **PARADIGM SHIFT**

If you've decided that a fundamental change in technology is in the best interests of the team and the organization, a rewrite is often the only option.

company may be making a strategic move from technologies such as Spring MVC to a more lightweight framework such as Ruby on Rails in the hope of improving developer productivity

The Third Way:
incremental rewrite



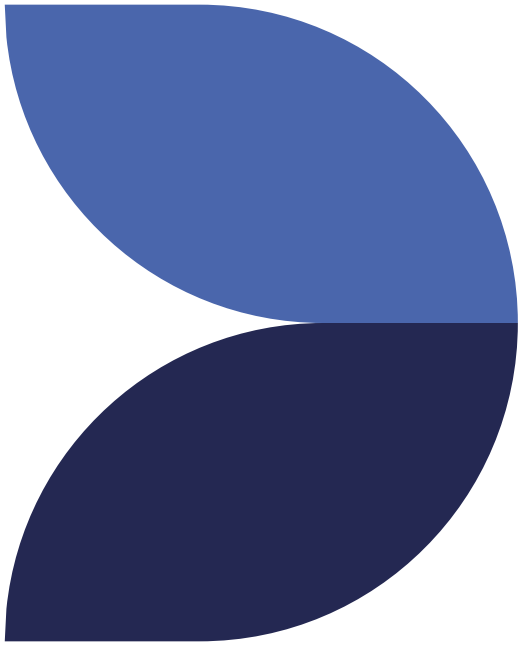
Sometimes you'll really want to rewrite at least some of an application, but a full rewrite carries too much risk

- The basic idea is to split the rewrite into a number of smaller phases, but it's important to stress two points:
- Each individual phase should provide business value.
- It should be possible to stop the project after any given phase, and still reap some benefits

One way to split up a rewrite is to divide the legacy software into a number of logical components and then rewrite them one at a time



Refactoring



Refactorings are changes to a program that improve its internal design but do not change its behavior.

Renaming a class that is used widely throughout a project could take a lot of time but the Eclipse refactoring wizard makes the job easier by automatically detecting all dependencies on the class being renamed and modifying them also.

Refactoring process using Eclipse with a Java

"Extract Method" refactoring



Step 1: Open Eclipse and Create a Java Project

Step 2: Write Sample Java Code

```
public class Calculator {
    public int add(int a, int b) {
        int result = a + b;
        return result;
    }
    public int multiply(int a, int b) {
        int result = a * b;
        return result;
    }
    public int calculate(int x, int y) {
        // Some complex calculations here
        int intermediateResult = add(x, y);
        int finalResult = multiply(intermediateResult, 2);
        return finalResult;
    }
    public static void main(String[] args) {
        Calculator calculator = new Calculator();
        int result = calculator.calculate(5, 3);
        System.out.println("Result: " + result);
    }
}
```

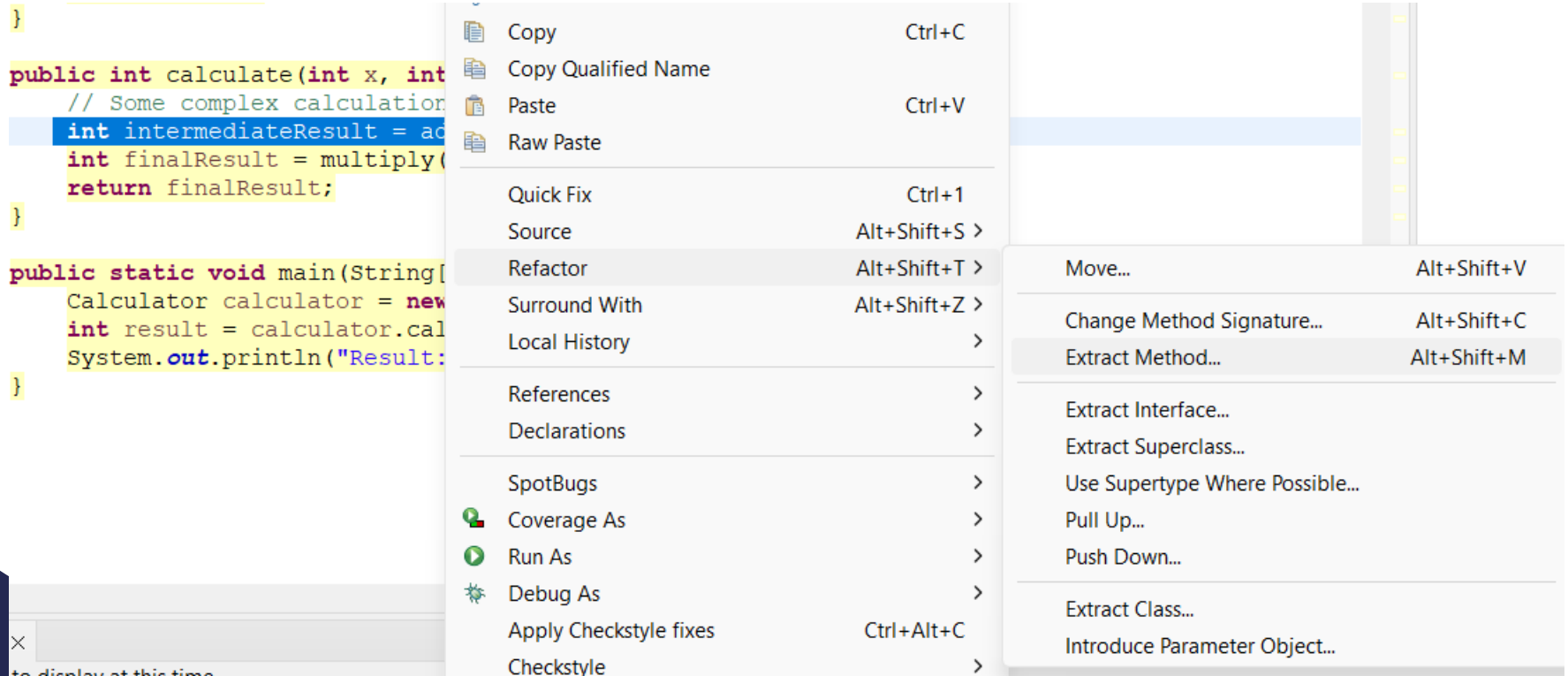
Now, let's say you want to refactor the calculate method by extracting a new method to calculate the intermediate result. Here's how you can do it using Eclipse:

1. Open Eclipse and load your Java project.
2. Locate the calculate method in your code, which contains the complex calculations.
3. Place the cursor inside the calculate method, specifically where the intermediate calculation is done:

int intermediateResult = add(x, y);

Right-click on this line of code and select "Refactor" from the context menu.

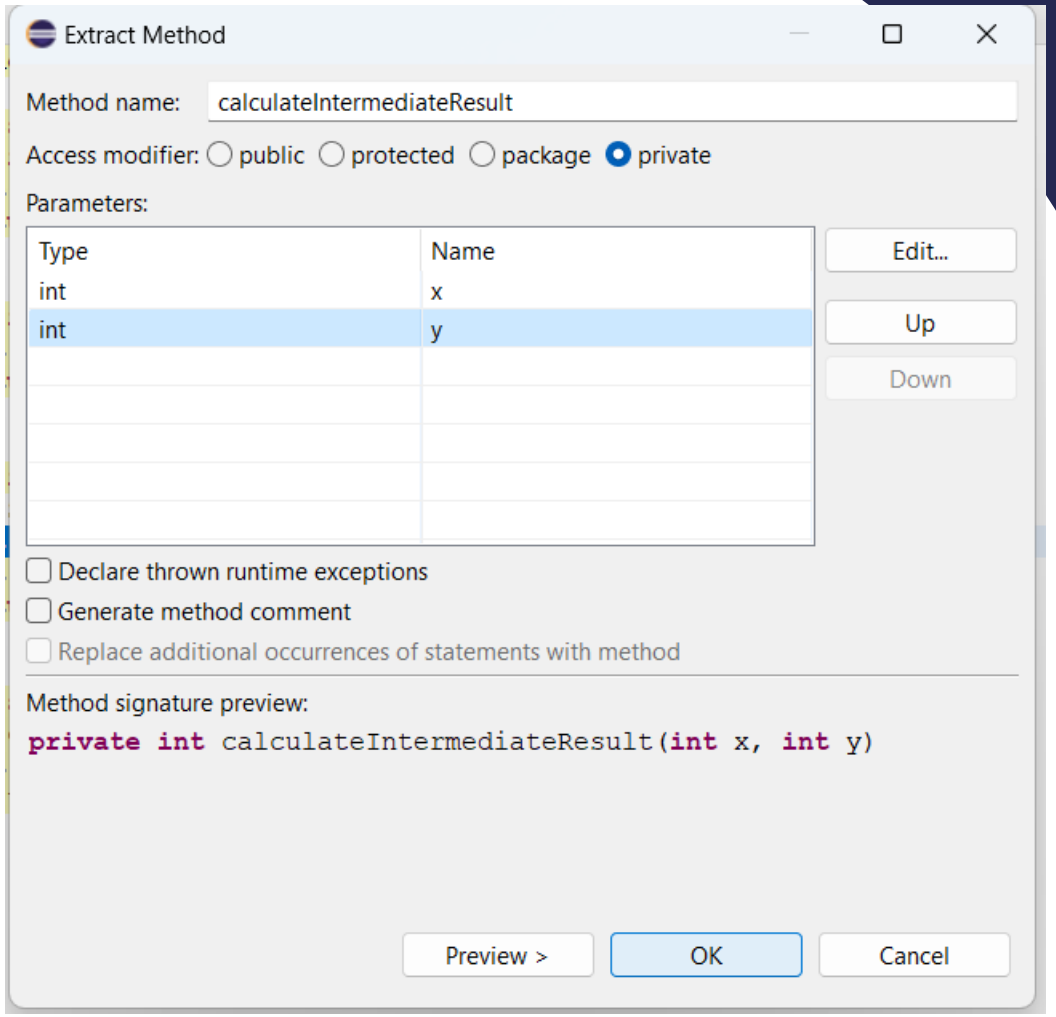
In the submenu, choose "Extract Method."



A dialog box will appear. Enter a name for the new method, let's call it calculateIntermediateResult.

You can choose to have Eclipse create parameters for the extracted method by selecting the appropriate variables, in this case, x and y. Make sure the checkbox for "Save as a method parameter" is checked.

Click the "OK" button to create the new method.



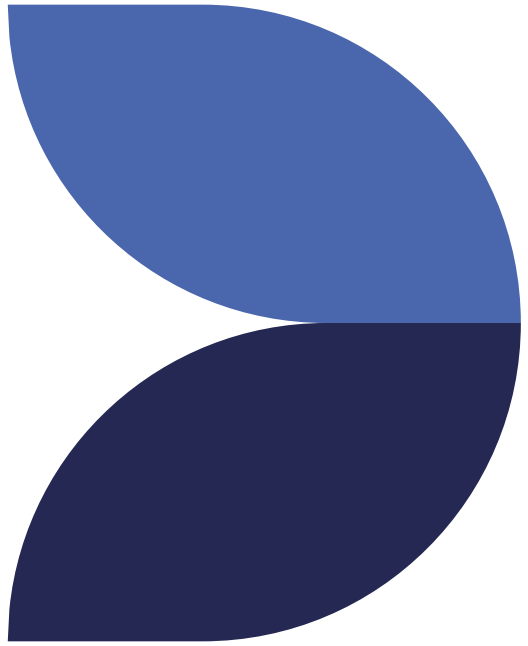
```java
package demo1;

public class Calculator {
public int add(int a, int b) {
int result = a + b;
return result;
}
public int multiply(int a, int b) {
int result = a * b;
return result;
}
public int calculate(int x, int y) {
// Some complex calculations here
int intermediateResult = calculateIntermediateResult(x, y);
int finalResult = multiply(intermediateResult, 2);
return finalResult;
}

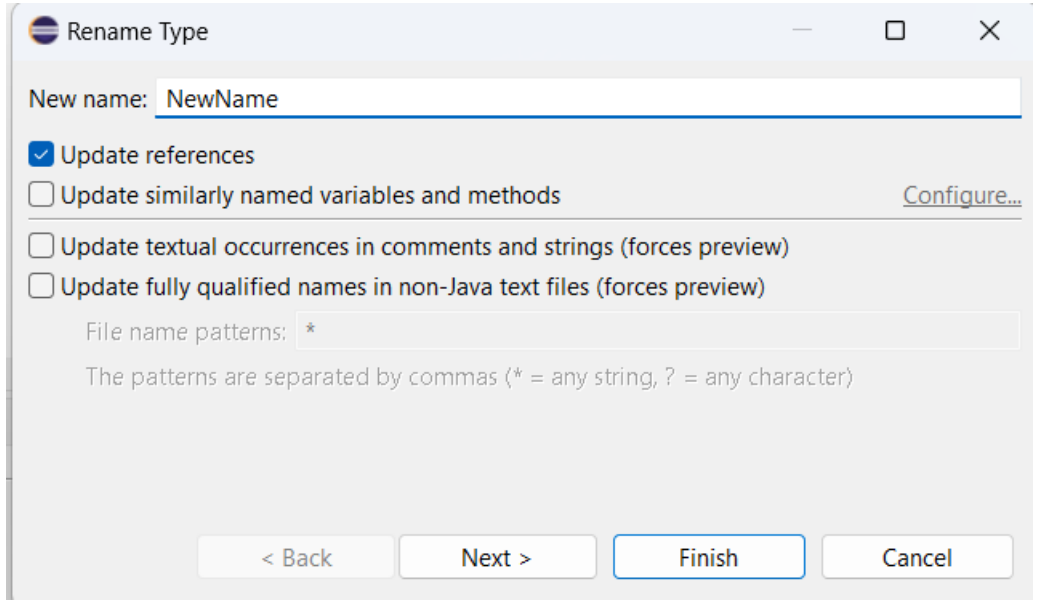
private int calculateIntermediateResult(int x, int y) {
int intermediateResult = add(x, y);
return intermediateResult;
}
public static void main(String[] args) {
Calculator calculator = new Calculator();
int result = calculator.calculate(5, 3);
System.out.println("Result: " + result);
}
}
```

You've successfully extracted a new method, calculateIntermediateResult, which encapsulates the intermediate calculation.

Rename



Rename: If you want to change the name of the type, select "Rename." You can then provide a new name for the type, and Eclipse will automatically update all references to the type within your code.



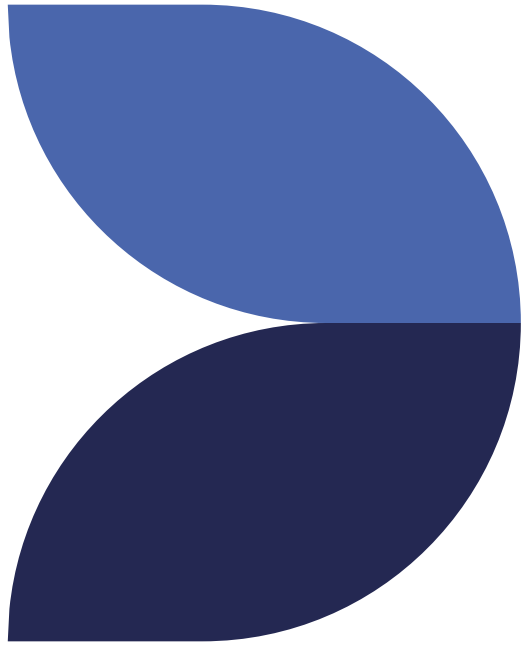

```
public class MyClass {
    private int varOne;
    private String varTwo;

    public MyClass(int a, String b) {
        this.varOne = a;
        this.varTwo = b;
    }

    public void printValues() {
        System.out.println("varOne: " + varOne);
        System.out.println("varTwo: " + varTwo);
    }

    public int calculateSum(int x, int y) {
        return x + y;
    }
}
```

Move



Move: If you want to move the type to a different package or source folder, select "Move." Eclipse will guide you through the process of selecting the destination.

demo1.java

```
package demo1;

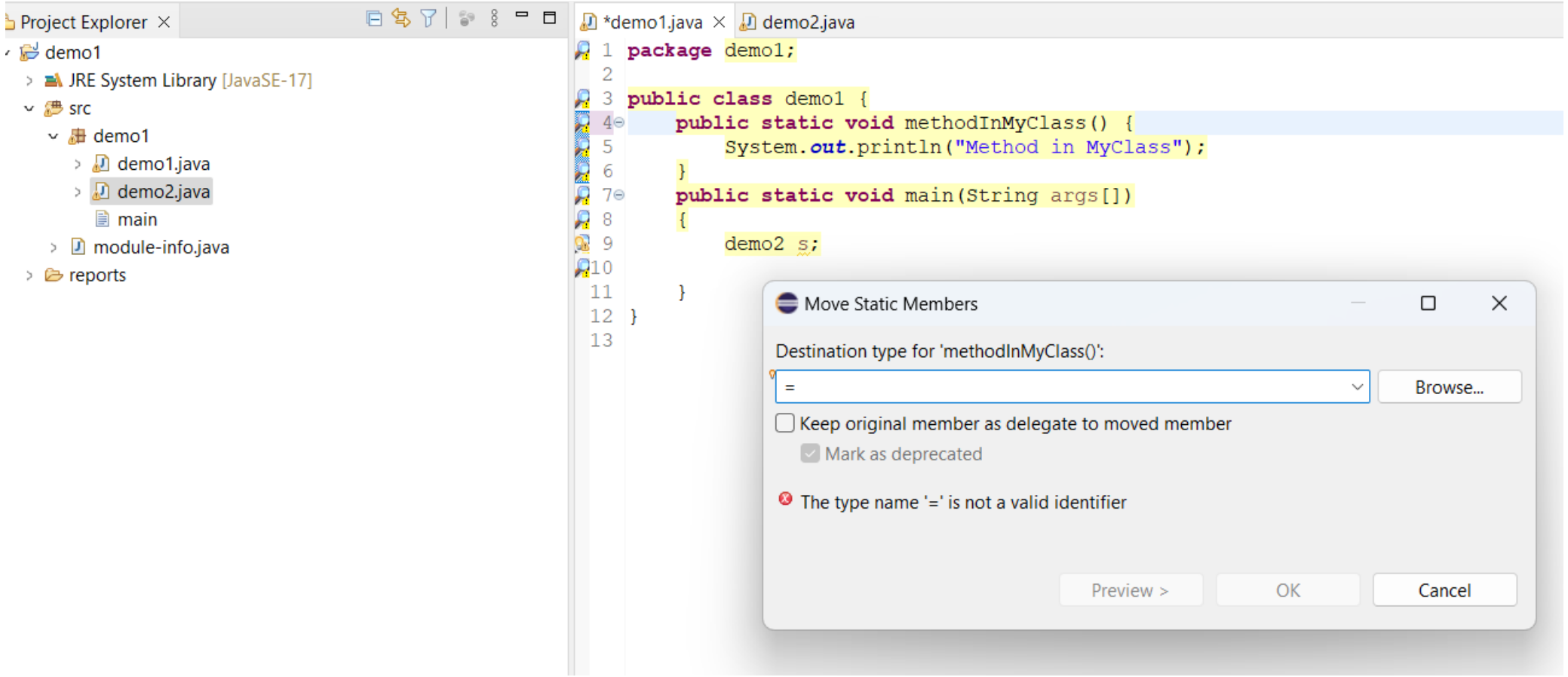

public class demo1 {
public static void methodInMyClass() {
System.out.println("Method in MyClass");
}
public static void main(String args[])
{
demo2 s;
}
}
```

demo2.java

```
package demo1;


public class demo2 {
public void methodInAnotherClass() {
System.out.println("Method in AnotherClass");
}


}
```


Project Explorer
demo1
JRE System Library [JavaSE-17]
src
demo1
demo1.java
demo2.java
main
module-info.java
reports
*demo1.java
demo2.java
1 package demo1;
2
3 public class demo1 {
4 public static void methodInMyClass() {
5 System.out.println("Method in MyClass");
6 }
7 public static void main(String args[])
8 {
9 demo2 s;
10
11 }
12 }
13
Move Static Members
Destination type for 'methodInMyClass()':
=
Browse...
Keep original member as delegate to moved member
Mark as deprecated
The type name '=' is not a valid identifier
Preview >
OK
Cancel

demo2.java

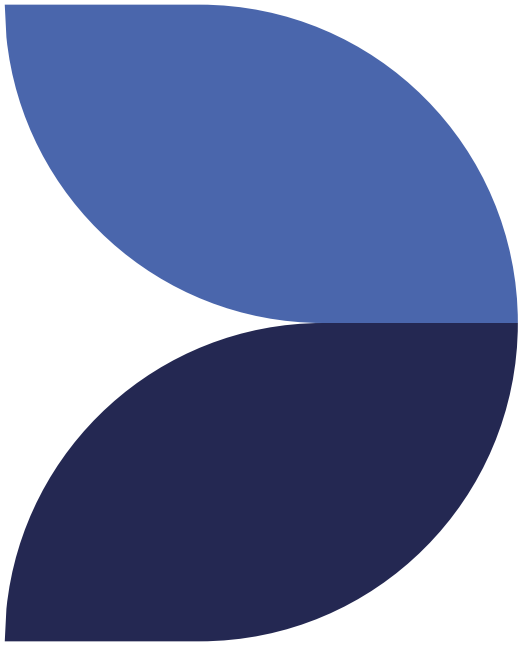
```
package demo1;


public class demo2 {
public void methodInAnotherClass() {
System.out.println("Method in
AnotherClass");
}


public static void methodInMyClass() {
System.out.println("Method in MyClass");
}


}
```

Extract method



Extract method

Problem
You have a code fragment that can be grouped together.

```
void printOwing() {
  printBanner();

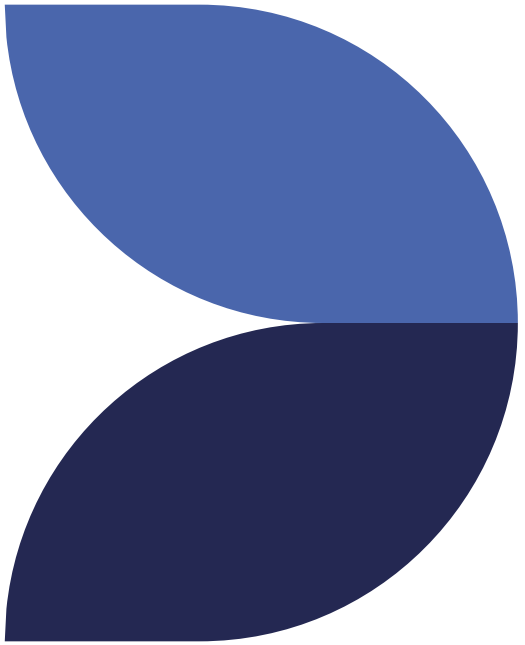
  // Print details.
  System.out.println("name: " + name);
  System.out.println("amount: " + getOutstanding());
}
```

Solution
Move this code to a separate new method (or function) and replace the old code with a call to the method.

```
void printOwing() {
  printBanner();
  printDetails(getOutstanding());
}

void printDetails(double outstanding) {
  System.out.println("name: " + name);
  System.out.println("amount: " + outstanding);
}
```

Extract variable



Extract variable

Problem
You have an expression that’s hard to understand.

```
void renderBanner() {
  if ((platform.toUpperCase().indexOf("MAC") > -1)
&&
       (browser.toUpperCase().indexOf("IE") > -1) &&
        wasInitialized() && resize > 0 )
  {
    // do something
  }
}
```

Solution
Place the result of the expression or its parts in separate variables that are self-explanatory.

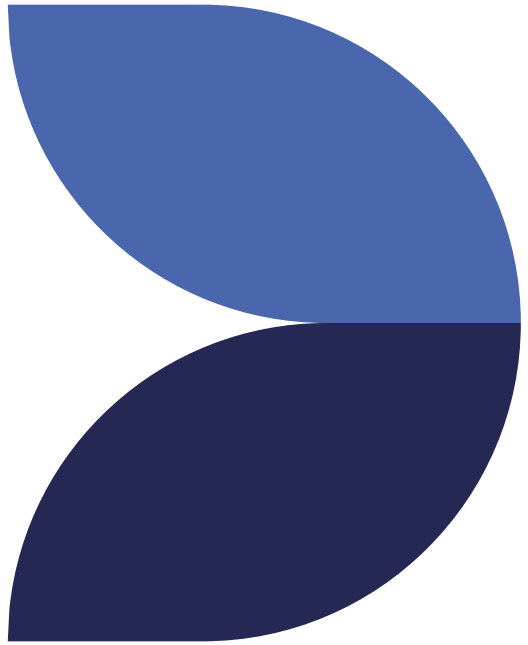
```
void renderBanner() {
  final boolean isMacOs =
platform.toUpperCase().indexOf("MAC") > -1;
  final boolean isIE = browser.toUpperCase().indexOf("IE") >
-1;
  final boolean wasResized = resize > 0;

  if (isMacOs && isIE && wasInitialized() && wasResized) {
    // do something
  }
}
```

The main reason for extracting variables is to make a complex expression more understandable, by dividing it into its intermediate parts.



Inline method



Inline methods

Problem: When a method body is more obvious than the method itself, use this technique.
Solution: Replace calls to the method with the method's content and delete the method itself.

Problem

When a method body is more obvious than the method itself, use this technique.

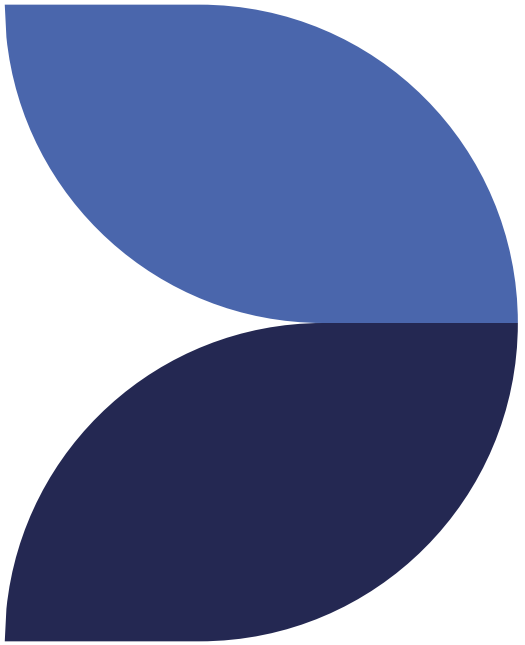
```
class PizzaDelivery {
  // ...
  int getRating() {
    return moreThanFiveLateDeliveries() ? 2 : 1;
  }
  boolean moreThanFiveLateDeliveries() {
    return numberOfLateDeliveries > 5;
  }
}
```

Solution

Replace calls to the method with the method's content and delete the method itself.

```
class PizzaDelivery {
  // ...
  int getRating() {
    return numberOfLateDeliveries > 5 ? 2 : 1;
  }
}
```

Inline Temp



Inline Temp

Problem
You have a temporary variable that's assigned the result of a simple expression and nothing more.

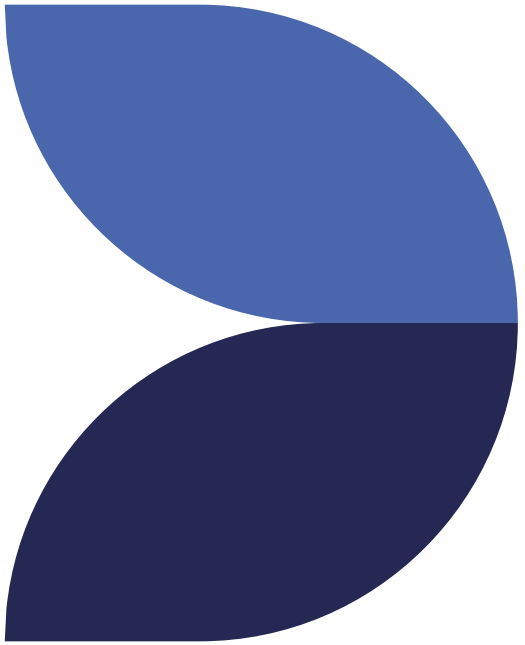
```
boolean hasDiscount(Order order) {
  double basePrice = order.basePrice();
  return basePrice > 1000;
}
```

Solution
Replace the references to the variable with the expression itself.

```
boolean hasDiscount(Order order) {
  return order.basePrice() > 1000;
}
```



Pull Up and Push Down



Pull Up and Push Down: These refactorings are used for moving methods and fields between superclasses and subclasses. "Pull Up" moves members from a subclass to a superclass, while "Push Down" moves members from a superclass to a subclass.

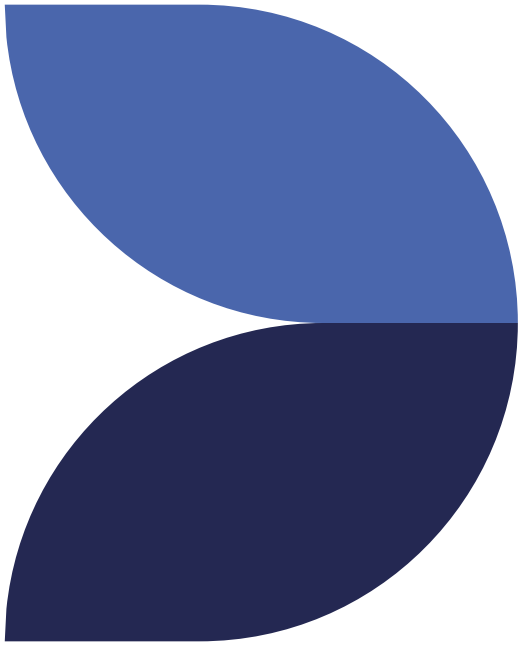
```
class Animal {
    // Common properties and methods for all animals.
}

class Dog extends Animal {
    public void bark() {
        System.out.println("Woof!");
    }
}

class Cat extends Animal {
    public void meow() {
        System.out.println("Meow!");
    }
}
```

pull up the meow method from Cat to the superclass Animal.

Extract interface



Extract Interface: If you want to create an interface from a class, select "Extract Interface." This can be useful for defining a contract that multiple classes need to implement.

- Extracting an interface is a common refactoring technique in Java used to define a new interface based on existing class methods.
- This allows you to abstract the behavior of one or more classes into a shared interface, promoting code flexibility and maintainability.

```
public class MyServiceImpl {
    public void methodOne() {
        // Implementation of methodOne
    }

    public void methodTwo() {
        // Implementation of methodTwo
    }
}
```

extract an interface from MyServiceImpl that includes methodOne and methodTwo.

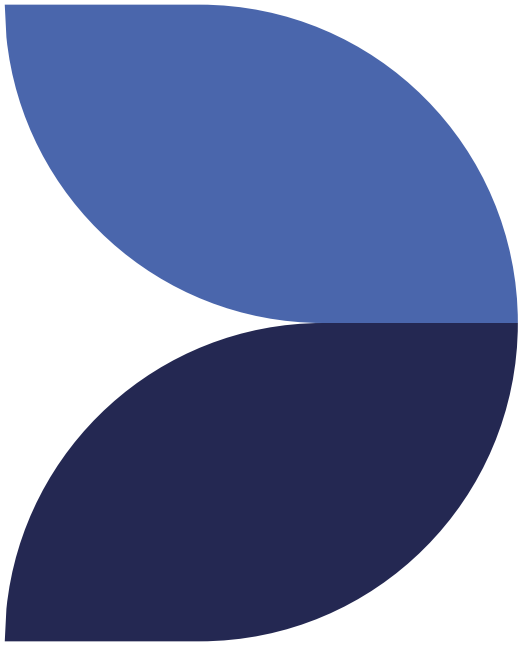
```
public interface MyService {
    void methodOne();

    void methodTwo();
}

public class MyServiceImpl implements MyService {
    public void methodOne() {
        // Implementation of methodOne
    }

    public void methodTwo() {
        // Implementation of methodTwo
    }
}
```

Extract Superclass



Extract Superclass: If you want to create a common superclass for several classes, select "Extract Superclass." This can help in reducing duplicated code.

```
public class Dog {
    private String name;
    private int age;

    public Dog(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void bark() {
        System.out.println("Woof!");
    }

    public void fetch() {
        System.out.println("Fetching the ball.");
    }
}
```

```
public class Cat {
    private String name;
    private int age;

    public Cat(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void meow() {
        System.out.println("Meow!");
    }

    public void sleep() {
        System.out.println("Sleeping...");
    }
}
```

Select one of the classes from which you want to extract a superclass (e.g., Dog).

Step 4: Right-click on the selected class and choose "Refactor" from the context menu, then select "Extract Superclass."

Step 5: A dialog or window will appear, allowing you to configure the extraction process. You can:

Specify the name of the new superclass (e.g., Animal).
Choose the attributes and methods you want to include in the superclass (select name, age, bark, and meow).
Step 6: Click the "OK" or "Finish" button to confirm the extraction.

Step 7: After confirming, your code will be updated as follows:

```
public class Animal {
    private String name;
    private int age;

    public Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void bark() {
        System.out.println("Woof!");
    }

    public void meow() {
        System.out.println("Meow!");
    }
}

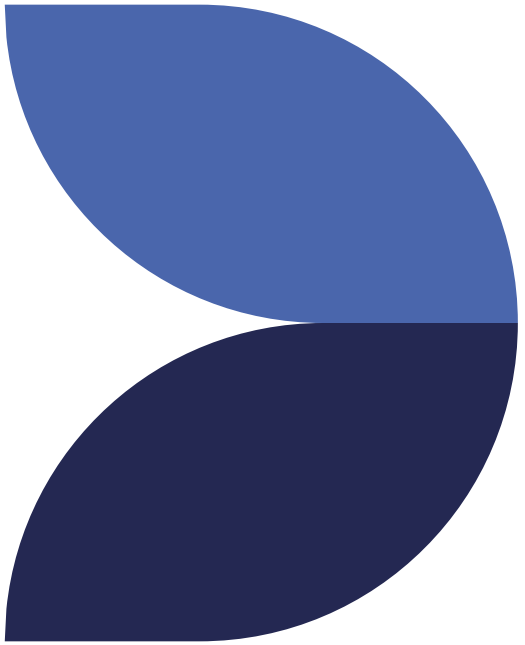
public class Dog extends Animal {
    public Dog(String name, int age) {
        super(name, age);
    }

    // Dog-specific methods
}
```

```
public class Cat extends Animal {
    public Cat(String name, int age) {
        super(name, age);
    }

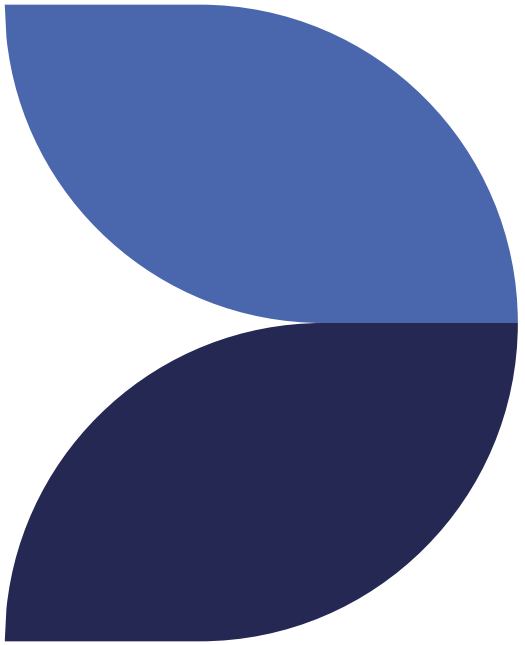
    // Cat-specific methods
}
```

Delete



Delete: If you want to remove a type from your project, select "Delete." Be careful when using this option, as it will permanently delete the type and all its references.

Replace Temp with Query



Problem

You place the result of an expression in a local variable for later use in your code.

```
double calculateTotal() {
  double basePrice = quantity * itemPrice;
  if (basePrice > 1000) {
    return basePrice * 0.95;
  }
  else {
    return basePrice * 0.98;
  }
}
```

Solution

Move the entire expression to a separate method and return the result from it. Query the method instead of using a variable. Incorporate the new method in other methods, if necessary.

```
double calculateTotal() {
  if (basePrice() > 1000) {
    return basePrice() * 0.95;
  }
  else {
    return basePrice() * 0.98;
  }
}
double basePrice() {
  return quantity * itemPrice;
}
```

Mikado Method

a structured way to make major changes to complex code



- four basic and well-known concepts that summarize the "process" of the Mikado Method:
  - *Set a goal,*
  - *Experiments,*
  - *Visualization, and*
  - *Undo (When an experiment for implementing a goal or a prerequisite has broken your system, and you have visualized what you need to change in the system to avoid it breaking, you want to your changes to restore a previously undo working state.).*
- It involves creating a "Mikado Graph" that visualizes the dependencies between various code changes and allows you to plan and execute refactorings incrementally.

```
public class ShoppingCart {
    private List<Product> products = new ArrayList<>();

    public void addProduct(Product product) {
        products.add(product);
    }

    public void removeProduct(Product product) {
        products.remove(product);
    }

    public double calculateTotalPrice() {
        double totalPrice = 0;
        for (Product product : products) {
            totalPrice += product.getPrice();
        }
        return totalPrice;
    }

    // Other methods...
}

public class Product {
    private String name;
    private double price;

    // Constructors, getters, and setters...
}
```

ShoppingCart Class:

The ShoppingCart class represents a shopping cart in an e-commerce system. It is responsible for managing a collection of Product objects and calculating the total price of the items in the cart, including any discounts.

Attributes:
products: A list that holds the products added to the shopping cart.
discountCalculator: An instance of the DiscountCalculator class used to calculate discounts.
Methods:
addProduct(Product product): Adds a product to the shopping cart.
removeProduct(Product product): Removes a product from the shopping cart.
calculateTotalPrice(): Calculates the total price of the products in the cart, factoring in any applicable discounts.
Other methods related to managing the shopping cart.

```
public class ShoppingCart {
    private List<Product> products = new ArrayList<>();

    public void addProduct(Product product) {
        products.add(product);
    }

    public void removeProduct(Product product) {
        products.remove(product);
    }

    public double calculateTotalPrice() {
        double totalPrice = 0;
        for (Product product : products) {
            totalPrice += product.getPrice();
        }
        return totalPrice;
    }

    // Other methods...
}

public class Product {
    private String name;
    private double price;

    // Constructors, getters, and setters...
}
```

Product Class:

The Product class represents a product available for purchase in the e-commerce system. It holds information about the product, such as its name and price.

Attributes:

productName: A string representing the name or description of the product.

productPrice: A double representing the price of the product.

Methods:

Constructors and methods for getting and setting the product's attributes (getters and setters).

Other methods as needed for product-related operations.

Mikado graph

Initial State
|
V
Task 1: Extract Discount Calculation
|
V
Task 2: Rename Product class fields
|
V
Task 3: Refactor Pricing Logic
|
V
Task 4: Optimize ShoppingCart class

Start with Task 1: Extract Discount Calculation:

Extract the discount calculation logic from the ShoppingCart class into a separate class called DiscountCalculator

```
public class DiscountCalculator {
    public double calculateDiscountedPrice(List<Product>
products) {
        double totalPrice = 0;
        for (Product product : products) {
            totalPrice += product.getPrice();
        }
        // Apply discount logic...
        return totalPrice;
    }
}
```

DiscountCalculator Class:

The DiscountCalculator class encapsulates the logic for calculating discounts on a collection of products. It is responsible for applying any discount rules to the products and determining the final discounted price.

Methods:

calculateDiscountedPrice(List<Product> products): Takes a list of products as input and calculates the total price after applying any applicable discounts. Additional methods related to discount calculation logic can be added here.

Update the ShoppingCart class to use the DiscountCalculator:

```
public class ShoppingCart {
    private List<Product> products = new ArrayList<>();
    private DiscountCalculator discountCalculator = new
DiscountCalculator();

    public void addProduct(Product product) {
        products.add(product);
    }

    public void removeProduct(Product product) {
        products.remove(product);
    }

    public double calculateTotalPrice() {
        return
discountCalculator.calculateDiscountedPrice(products);
    }

    // Other methods...
}
```

Mark Task 1 as Completed:

Initial State
|
V
[X] Task 1: Extract Discount Calculation
|
V
Task 2: Rename Product class fields
|
V
Task 3: Refactor Pricing Logic
|
V
Task 4: Optimize ShoppingCart class

Proceed with Task 2: Rename Product Class Fields:
Rename the name and price fields in the Product class:

```
public class Product {
    private String productName;
    private double productPrice;

    // Constructors, getters, and setters...
}
```

Update any references to these fields in other parts of the codebase.

Mark Task 2 as Completed:

Initial State
|
V
[X] Task 1: Extract Discount Calculation
|
V
[X] Task 2: Rename Product class fields
|
V
Task 3: Refactor Pricing Logic
|
V
Task 4: Optimize ShoppingCart class

Initial State
|
V
[X] Task 1: Extract Discount Calculation
|
V
[X] Task 2: Rename Product class fields
|
V
Task 3: Refactor Pricing Logic
|
V
Task 4: Optimize ShoppingCart class (This task could involve various improvements, such as removing redundant code, improving data structures, or enhancing error handling.)
Task 5: Introduce Customer class

Continue with the Remaining Tasks: Continue with Tasks 3 and 4, and mark them as completed as you finish each one.

Simplify price logic

```
public void applyDiscount(double percentage) {
        double discountAmount = (percentage / 100) *
totalPrice;
        discount += discountAmount;
        totalPrice -= discountAmount;
    }
```

```
import java.util.ArrayList;
import java.util.List;

public class ShoppingCart {
    private List<Product> products = new ArrayList<>();
    private double totalPrice = 0;
    private double discount = 0;

    public void addProduct(Product product) {
        products.add(product);
        totalPrice += product.getPrice();
    }

    public void applyDiscount(double percentage) {
        double discountAmount = (percentage / 100) * totalPrice;
        discount += discountAmount;
        totalPrice -= discountAmount;
    }

    public double calculateTotalPrice() {
        return totalPrice;
    }

    public double getDiscount() {
        return discount;
    }

    // Other methods...

    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart();
        Product product1 = new Product("Item 1", 20.0);
        Product product2 = new Product("Item 2", 30.0);

        cart.addProduct(product1);
        cart.addProduct(product2);

        cart.applyDiscount(10); // Apply a 10% discount

        System.out.println("Total Price: $" + cart.calculateTotalPrice());
        System.out.println("Discount Applied: $" + cart.getDiscount());
    }
}

class Product {
    private String name;
    private double price;

    public Product(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public double getPrice() {
        return price;
    }
}
```

By following this approach, you can systematically refactor your e-commerce codebase while keeping track of dependencies and maintaining a working state throughout the process.

Mandatory readings

- Chapter 3 and 4, Chris Birchall - Re-Engineering Legacy Software-Manning Publications (2016)



Thank you